SOFUU 素風 · V0.2.0-BETA · UPDATED ARCHITECTURE REFERENCE

RUNTIME ARCHITECTURE MAP

Every layer and subsystem of the Sofuu runtime — from the **C/C++ core & SIMD kernels** (QuickJS, libuv, NEON/AVX2, QTSQ) through the **Rust engine** with its **in-process Edge LLM inference**, **CMA cognitive memory**, and **five decision ML models**, up to the **shipped JavaScript drivers** and native **macOS Swift & Android Kotlin apps**.

OCTOBER 2026
BUILT FROM SOURCE · AI-NATIVE-JS-RUNTIME

5 LAYERS · 6 SURFACES
IN-PROCESS INFERENCE · 4.2 MB RELEASE

OVERVIEW

The Big Picture

Sofuu is an embeddable in-process AI runtime combining sub-millisecond JavaScript execution, hardware-saturated local LLM inference, autonomous agent loops, and persistent cognitive vector memory into a single standalone binary.

4.2MB	~3ms	111.3k	141.5tk/s
STRIPPED RELEASE BINARY	COLD JS STARTUP	LINES OF CORE RUST	ARM64 PEAK PREFILL

THE LAYER STACK — TOP-LEVEL HIERARCHY

LAYER 4 — Surfaces
Native host apps & user interfaces

`CLI / TUI chat` · `sofuu run file.ts|.js` · `C-ABI Funnel (embed)` · `macOS IDE (Swift)` · `Android App (Kotlin)` · `Tauri Desktop`

↓

LAYER 3 — Shipped JS
5 drivers embedded into the binary

`agent.js 2,160 ln` · `chat.js 1,787 ln` · `tools.js 428 ln` · `web.js 327 ln` · `rlm.js 305 ln`

↓

LAYER 2 — Rust core
crates/sofuu-core · 115 files · 111.3k LOC

`rt/ loop substrate (20)` · `infer/ edge LLM engine (28)` · `memory/ CMA 4-tier + HNSW` · `ml/ 5 tiny decision models` · `mcp/ + rlm/ + http/` · `chat.rs + session mesh`

↓

LAYER 1 — FFI bridge
crates/sofuu-ffi + sofuu-capi

`qjs.rs QuickJS` · `uv.rs libuv` · `curl.rs libcurl` · `qtsq.rs codec` · `capi headless C funnel`

↓

LAYER 0 — C core & SIMD
deps/ + src/simd/ + QTSQ store format

`quickjs JS engine` · `libuv event loop` · `llhttp HTTP parse` · `SIMD NEON / AVX2 kernels` · `QTSQ encrypted store`

invariants Single standalone binary ≤ 5.0 MB (4.2 MB stripped) · Zero daemon dependencies · Zero-copy mmap tensor streaming · AES-256-GCM memory encryption · 100% automated test verification.

VISUAL HIERARCHY

Every Part, One Map



☐ Rust-owned part ☐ C/C++ or boundary part

`infer/ (28)` = 28 source files in the subsystem

C/C++ Foundation, SIMD & FFI Funnel

Layer 0 and Layer 1. Bare-metal hardware acceleration, the C core, safe Rust facades, and the headless C-ABI funnel that lets mobile apps, desktop IDEs, and host languages embed the entire engine in-process.

PART	SUB-PART	ROLE IN THE RUNTIME
DEPS/ & SRC/SIMD/ — HARDWARE PRIMITIVES, C CORE & KERNELS (LAYER 0)		
`deps/quickjs`	JS engine	Lightweight ES2023+ JavaScript engine executing all user code, agent loops, and sandboxed tools with minimal memory.
`deps/libuv`	event loop	Async I/O: timers, sockets, pipes, subprocesses. Interleaved with QuickJS microtasks after every poll tick.
`deps/llhttp`	HTTP parsing	Zero-copy request/response parser backing the embedded HTTP server and streaming fetch client.
`src/simd/infer_kernels.cpp`	SIMD GEMM / GEMV	Hand-tuned NEON and AVX2 matrix math for q4_K, q8_0, q8_K, and PQ2_0; multi-threaded tile worker pool.
`src/simd/platform_affinity.h`	CPU core pinning	Hardware affinity governor: binds inference threads to performance cores on Android (cpufreq) and Apple Silicon QoS.
`QTSQ container codec`	encrypted binary store	Append-only, checksummed, segmented container format with AES-256-GCM encryption for brain and session storage.
CRATES/SOFUU-FFI — SAFE RUST BINDINGS TO C CORE (LAYER 1)		
`src/qjs.rs / qjs_rt.rs`	QuickJS & sandbox	Safe Rust wrapper over QuickJS context, memory quotas, exception draining, and isolated RLM sandboxes.
`src/uv.rs`	libuv bindings	Safe event-loop handle bindings backing all async Rust-side timers and polling.
`src/curl.rs`	libcurl transport	UAF-hardened HTTP streaming transport backing fetch() and cloud LLM streaming.
`src/qtsq.rs`	store codec	Safe container read/write/compact bindings powering the Cognitive Memory Architecture.
CRATES/SOFUU-CAPI — UNIVERSAL HEADLESS C-ABI FUNNEL (LAYER 1)		
`sofuu_rt_new / free`	runtime lifecycle	Instantiates and frees hermetic engine instances with isolated configs, memory limits, and thread handles.
`sofuu_call / stream`	universal dispatch	Thread-safe synchronous and streaming dispatch with cancellation tokens; powers native host calls.
`MEMORY_FACADE_JS`	CMA headless facade	Exposes all 16 CMA methods with handle tombstoning (slot reuse), hybrid BM25 query text, and encrypted persistence.

The Runtime Substrate — `sofuu-core/src/rt/`

Twenty files that coordinate the event loop, expose Posix I/O primitives to JavaScript, route streaming LLM calls, and enforce model capability limits with dynamic discovery ladders.

SUB-PART (FILE)	ROLE IN THE RUNTIME SUBSTRATE
`engine.rs / shipped.rs`	QuickJS engine lifecycle and AOT precompiled bytecode snapshotting (7 shipped `.bc` drivers) yielding sub-16ms cold start.
`loop.rs / timer.rs`	Global libuv event loop driver with bounded microtasks (N=1024), W3C `scheduler.yield()`, and starvation protection.
`zerocopy.rs`	64-byte aligned SIMD memory bridge (`sofuu.memory/buffer`): zero-copy `Float32Array` views and safe detachment.
`async_context.rs`	Native `AsyncLocalStorage` & `sofuu.context`: propagates traceId, sessionId, and AbortSignal across async chains with 0 crosstalk.
`sandbox.rs`	Hermetic agent sandbox with single-instruction fuel metering via QuickJS interrupt counters, ≤6ms deadlines, and memory caps.
`ai.rs`	The `sofuu.ai` streaming router: dispatches across OpenAI, Anthropic, Ollama, and embedded local inference.
`memory.rs`	CMA JavaScript module: features `cma_vec_view` accepting both Float32Array (zero-copy) and standard JS arrays.
`promise.rs`	Promise ↔ libuv async bridge; async context capture, unhandled-rejection tracking, and Rust future bridging.
`fs.rs / process_spawn.rs`	Asynchronous filesystem operations (`sofuu.fs`) and streaming child subprocess execution (`sofuu.spawn/exec`).
`http_client.rs / server.rs`	High-throughput `fetch()` backed by UAF-hardened libcurl and embedded async `sofuu.http` HTTP/1.1 server.
`http_sse.rs`	JS-facing `sofuu.SSEParser` streaming parser over incoming server-sent event streams.
`mcp.rs`	Model Context Protocol JS module: client/server lifecycle, tool calling, and resource indexing.
`model_caps.rs`	Authoritative model capability registry & dynamic HTTP 400 discovery ladder (context windows, thinking tokens).
`host_poke.rs`	Thread-safe cross-thread wakeup pipe: lets host UI threads inject events into a running JS loop safely.
`session_js.rs`	Synchronous session-mesh primitives exposed to shipped JS for workspace context sharing.
`npm.rs / cjs.rs`	Lightweight npm package resolution and CommonJS compatibility shims for legacy modules.
`tui.rs`	Minimal alt-screen TUI driver powering interactive CLI chat sessions (`sofuu chat`).

The Edge Inference Engine — `sofuu-core/src/infer/`

Twenty-eight files delivering high-throughput local LLM execution directly in-process with zero external daemons, supporting Apple Metal GPU acceleration, CPU SIMD, speculative lookahead, and quantized KV caches.

SUB-PART (FILE)	ARCHITECTURE	ROLE IN LOCAL INFERENCE
`model.rs`	Block Graph	Composable Transformer Block Graph: executes LLaMA 1/2/3, Qwen 2/2.5/3, Gemma 1/2, Mistral, SmolLM.
`metal.rs`	Apple Metal	Metal GPU decode backend: unified zero-copy KV cache with single-commit-per-layer batches eliminating GPU stalls.
`quant.rs`	Quantization	SIMD dequantization and GEMV kernels: q4_K, q8_0, q8_K, PQ2_0 (2-bit), PTQ1_0 with 8-act panel sharing.
`lookahead.rs`	Speculative Tree	Multi-branch Jacobi tree speculation: boosts decode speed 1.5x–2.2x (up to 60.3 tk/s on Qwen, 20.1 tk/s on Pi 5).
`kivi.rs`	KIVI / TurboKV	Asymmetric 2-bit/4-bit quantized KV cache with rolling F32 residual window; reduces memory bandwidth 4x–8x.
`continuous.rs`	Continuous Batch	Continuous batching scheduler: concurrent request handling with decoupled prefill and decode phase metrics.
`dynamic_ffn.rs`	Sparse FFN	Activation energy gating: prunes 70%+ of SwiGLU down-projection weights dynamically for faster prefill.
`snapkv.rs`	Observation Pool	Long-context attention observation pooling: prunes middle prompt KV cache while preserving critical anchor tokens.
`weights.rs / mmap.rs`	Zero-Copy Ingestion	Non-temporal streaming weight loaders (MADV_SEQUENTIAL); ingests multi-gigabyte models in 17–38 ms.
`lfm2.rs / pytorch.rs`	Hybrid & Universal	Liquid LFM-2 conv/attention hybrid architecture; zero-copy ingestion of PyTorch (.pt/.pth) and NumPy (.npy) weights.

Cognitive Memory, Protocols & Core Subsystems

The cognitive memory architecture (CMA), approximate nearest-neighbor vector search, recursive language model sandboxing, and project-scoped real-time session synchronization.

SUB-PART	MODULE	ROLE IN THE RUNTIME
MEMORY/ — COGNITIVE MEMORY ARCHITECTURE (CMA) & HNSW INDEX		
`memory/cma.rs`	CMA Engine	4-tier human-like memory model: Working Set (B6 session), Episodic turns, Semantic facts, and Archive storage.
`memory/hnsw.rs`	HNSW Index	Hierarchical Navigable Small World vector search graph providing sub-millisecond approximate nearest neighbor retrieval.
`memory/recall_eval.rs`	Hybrid Re-Ranking	Candidate oversampling ((top_k * 3).max(6)) paired with BM25 rare-token re-ranking for needle precision when k < 5.
MCP/ & RLM/ — MODEL CONTEXT PROTOCOL & RECURSIVE LANGUAGE MODEL		
`mcp/mod.rs & jsonrpc.rs`	MCP Engine	Full Model Context Protocol client & server over JSON-RPC 2.0; parses tool calls, prompts, and resources.
`rlm/sandbox.rs`	Hermetic Sandbox	Nested, isolated QuickJS runtime: executes code sub-tasks in an isolated context where history lives as pure data.
`rlm/episode.rs & router.rs`	RLM Loop	Episode state machine routing complex multi-step reasoning turns into sandboxed execution with logging.
TOP-LEVEL DRIVERS & SYSTEM TOOLS		
`chat.rs`	Turn Driver	Rust interactive turn engine: streaming token chunking, automatic compaction, provider failover, and resume.
`session.rs`	Session Mesh	Multi-session real-time context sharing scoped to a project repository git root.
`bundler.rs & ts.rs`	Bundler & Stripper	Zero-dependency ESM bundler resolving import graphs into a single bundle; fast TypeScript type stripper.
`doctor.rs`	Diagnostic Suite	System health check: inspects hardware acceleration, QTSQ store integrity, vector dimensions, and journal state.

The ML Decision Suite — Five Tiny On-Device Models

Five specialized ~8.5k-parameter neural networks trained offline with `ml-train`, baked into the binary, running in ~30 microseconds to guide memory allocation, compaction, and tool safety.

MODEL 1
ml/freshness
Scores staleness and temporal relevance of recalled context before injection into the prompt.

GATE
ml/relevance
Pre-retrieval relevance gate: decides whether a memory query has sufficient signal to fetch.

GATE
ml/supervisor
Real-time tool supervisor: detects redundant or looping tool calls with confidence-weighted online learning.

MODEL 4
ml/compaction
Learned compaction trigger: replaces fixed token threshold cliffs with an intelligent summarization point.

MODEL 5 · CONTEXT GOVERNOR
ml/alloc
Context-window allocator: inspects model capabilities (registry → discovered → defaults), allocating per turn: compact point, tool result cap, recall budget, attach budget, and output reserve with mechanical Layer-0 clamp guarantees.

SUB-PART (FILE)	ROLE IN THE DECISION SUITE
`ml/net.rs`	Generic tiny feedforward MLP evaluator (24 → 96 → 40 → 1 shape family) executing in ~30 microseconds.
`ml/online.rs`	Online adaptation of the supervisor's output layer: confidence-weighted gradient updates with gold standard gates.
`ml/context.rs`	Shared in-memory context working set providing a structured representation of the active agent turn state.
`ml/*/features.rs`	Feature extractors computing 24+ numeric signals from model capabilities, session history, and task shape.
`ml/*/model.rs`	Per-family model wrapper loading baked weights via `include_bytes!` and executing inference.
`crates/ml-train`	Offline trainer: synthetic trajectory generators, channel-coverage splits, and automated weight baking.

Shipped JavaScript Drivers & Native Host Apps

Layer 3 embeds five JS drivers straight into the binary; Layer 4 provides first-class native host applications for macOS (Swift/SwiftUI), Android (Kotlin/Compose), and desktop Tauri.

SRC/JS/ — SHIPPED DRIVERS (EMBEDDED VIA INCLUDE_STR!)

DRIVER	LINES	ROLE IN THE RUNTIME
`agent.js`	2,160	Agent runtime: agent definitions, tool execution loop, multi-step planning, sub-agent spawning, trace events.
`chat.js`	1,787	Chat turn engine: token streaming, pre-flight allocation ladders, provider failover, conversation auto-resume.
`tools.js`	428	Native built-in tools: read, write, edit, grep, glob, list_dir, and sandboxed bash execution.
`web.js`	327	Web capabilities: keyless DuckDuckGo web search and clean readable text extraction from URLs.
`rlm.js`	305	RLM protocol harness running inside hermetic QuickJS sandboxes for recursive code execution.

NATIVE HOST APPLICATIONS (LAYER 4)

APPLICATION	STACK	CAPABILITIES & ARCHITECTURAL ROLE
`sofuu-macos/`	Swift · SwiftUI	Native macOS IDE: full file tree manager, gutter v3 (draw-time line arithmetic), debounced multi-file auto-save, direct C-API linking, and macOS Keychain API key protection.
`sofuu-android/`	Kotlin · Compose	Native Android App: direct JNI bridge (jni_bridge.c), big.LITTLE core affinity pinning (platform_affinity.h), streaming chat UI bubbles, procedural WebGL cloud background.
`sofuu-desktop/`	Tauri · React	Cross-platform desktop application: dedicated background engine thread, allow/deny permission gates, liquid-glass UI.

Support Crates, Storage, Seams & Invariants

SUPPORT CRATES & VERIFICATION SUITES

PART	SCOPE	ROLE
crates/ml-train	train.rs + 5 generators	Offline trainer generating synthetic trajectories with channel-coverage validation splits; bakes weights for ML suite.
crates/fuzz	7 cargo-fuzz targets	Continuous fuzz testing over trust boundaries: capi_call, jsonrpc, npm_spec, rlm_reply, rlm_snippet, sse, ts_strip.
Integration Tests	Automated Suites	92 core memory tests, 47 C-API tests, 62 inference tests, and 49/49 extreme torture tests (AES-256-GCM verified).

HOW THE PIECES CONNECT — THE THREE SEAMS

SEAM	WHAT CROSSES IT
C/C++ ↔ Rust	sofuu-ffi: QuickJS value marshalling, libuv loop polling ticks, libcurl transfers, QTSQ container operations.
Rust ↔ JS	rt/engine.rs registers every native sofuu.* module; shipped.rs embeds the 5 JS drivers; host_poke wakes the engine loop.
Engine ↔ Host Apps	sofuu-capi C-ABI: host_tool execution, stream events, stable handle tombstoning across Android JNI, Swift, and Tauri.

ARCHITECTURAL INVARIANTS & RELEASE BUDGET

INVARIANT	GUARANTEE
Release Binary Budget	Strictly ≤ 5.0 MB release binary budget (currently 4.2 MB stripped with complete inference and memory engines).
Data Privacy at Rest	Zero-leak AES-256-GCM authenticated encryption on disk; automatic hardware key derivation; no plaintext leaks.
In-Process Execution	Zero external background daemons; zero localhost network round-trips; instant < 4ms cold start.

reading order Start with The Big Picture (p.2) and Visual Hierarchy (p.3), then follow the tables upward: C/SIMD foundation (p.4), runtime substrate (p.5), edge inference engine (p.6), cognitive memory (p.7), decision ML suite (p.8), and native applications (p.9).